```json
{
"name": "DLQ_Manager",
"nodes": [
{
"parameters": {
"path": "dlq/add",
"httpMethod": "POST",
"responseMode": "lastNode",
"options": {}
},
"name": "Add to DLQ Webhook",
"type": "n8n-nodes-base.webhook",
"typeVersion": 2.1,
"position": [0, 0],
"webhookId": "dlq-add",
"id": "dlq_add_webhook_001"
},
{
"parameters": {
"jsCode": "// Validate and format DLQ input\nconst input = $input.first().json;\n\n//
Required fields\nconst required = ['failure_reason', 'original_payload'];\nfor (const field
of required) {\n  if (!input[field]) {\n    throw new Error(`Missing required field:
${field}`);\n  }\n}\n\nreturn [{\n  json: {\n    booking_id: input.booking_id || null,\n
idempotency_key: input.idempotency_key || null,\n    provider_id: input.provider_id || 1,\n
service_id: input.service_id || 1,\n    start_time: input.start_time || null,\n
customer_id: input.customer_id || input.chat_id || null,\n    chat_id: input.chat_id ||
null,\n    failure_reason: input.failure_reason,\n    last_error_message: input.error_message
|| input.failure_reason,\n    last_error_stack: input.error_stack || null,\n
original_payload: input.original_payload,\n    context_data: input.context_data || {\n
workflow_id: input.workflow_id || 'UNKNOWN',\n      execution_id: input.execution_id ||
'MANUAL',\n      failed_at: input.failed_at || new Date().toISOString()\n    }\n  }\n}];",
"options": {}
},
"name": "Validate DLQ Input",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [250, 0],
"id": "dlq_validate_002"
},
{
"parameters": {
"operation": "executeQuery",
"query": "SELECT booking_dlq_add($1::int, $2::text, $3::int, $4::int, $5::timestamp,
$6::text, $7::text, $8::text, $9::text, $10::text, $11::jsonb, $12::jsonb);",
"options": {
"queryReplacement": "={{ $json.booking_id }}, {{ $json.idempotency_key }}, {{
$json.provider_id }}, {{ $json.service_id }}, {{ $json.start_time }}, {{ $json.customer_id
}}, {{ $json.chat_id }}, {{ $json.failure_reason }}, {{ $json.last_error_message }}, {{
$json.last_error_stack || '' }}, {{ JSON.stringify($json.original_payload) }}, {{
JSON.stringify($json.context_data) }}"
}
},
"name": "Add to DLQ DB",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [500, 0],
"credentials": {
```

```
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "dlq_db_add_003"
},
{
"parameters": {
"jsCode": "// Return DLQ addition result\nconst dbResult = $('Add to DLQ
DB').first().json;\nconst input = $('Validate DLQ Input').first().json;\n\nreturn [{\n  json:
{\n    success: true,\n    error_code: null,\n    error_message: null,\n    data: {\n
dlq_id: dbResult.booking_dlq_add,\n      added_to_dlq: true,\n      failure_count: 1, // Will
be updated by DB function if duplicate\n      next_retry_at: new Date(Date.now() + 2 *
60000).toISOString(), // 2 minutes\n      message: `Booking added to Dead Letter Queue. Will
retry in 2 minutes.`,\n      idempotency_key: input.idempotency_key\n    },\n    _meta: {\n
source: 'DLQ_MANAGER',\n      timestamp: new Date().toISOString(),\n      version: '1.0.0'\n
}\n  }\n}];",
"options": {}
},
"name": "DLQ Add Response",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [750, 0],
"id": "dlq_add_response_004"
},
{
"parameters": {
"path": "dlq/status",
"httpMethod": "GET",
"responseMode": "lastNode",
"options": {}
},
"name": "DLQ Status Webhook",
"type": "n8n-nodes-base.webhook",
"typeVersion": 2.1,
"position": [0, 300],
"webhookId": "dlq-status",
"id": "dlq_status_webhook_005"
},
{
"parameters": {
"operation": "executeQuery",
"query": "SELECT * FROM v_booking_dlq_status;"
},
"name": "Get DLQ Status",
"type": "n8n-nodes-base.postgres",
```

```
"typeVersion": 2.6,
"position": [250, 300],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "dlq_status_db_006"
},
```

{
"parameters": {
"jsCode": "// Return DLQ status summary\nconst status = $('Get DLQ Status').all();\n\nconst summary = status.reduce((acc, item) => {\n acc[item.json.status] = {\n   count: item.json.count,\n   oldest: item.json.oldest_failure,\n   avg_failures: parseFloat(item.json.avg_failure_count || 0).toFixed(2)\n };\n return acc;\n}, {});\n\nreturn [{\n json: {\n   success: true,\n   error_code: null,\n   error_message: null,\n   data: {\n   summary: summary,\n   total_items: status.reduce((sum, item) => sum + (item.json.count || 0), 0),\n   generated_at: new Date().toISOString()\n   },\n _meta: {\n   source: 'DLQ_MANAGER',\n   timestamp: new Date().toISOString(),\n version: '1.0.0'\n   }\n }\n}];",
"options": {}
},
"name": "DLQ Status Response",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [500, 300],
"id": "dlq_status_response_007"
}
],
"connections": {
"Add to DLQ Webhook": {
"main": [
[
{
"node": "Validate DLQ Input",
"type": "main",
"index": 0
}
]
]
},
"Validate DLQ Input": {
"main": [
[
{
"node": "Add to DLQ DB",
"type": "main",
"index": 0
}
]
]
},
"Add to DLQ DB": {
"main": [
[
{
"node": "DLQ Add Response",
"type": "main",
"index": 0
}
]
]
},
"DLQ Status Webhook": {
"main": [
[
{

```
"node": "Get DLQ Status",
"type": "main",
"index": 0
}
]
]
},
"Get DLQ Status": {
"main": [
[
{
"node": "DLQ Status Response",
"type": "main",
"index": 0
}
]
]
}
},
"settings": {
```

```
"executionOrder": "v1"
},
"active": false,
"versionId": "dlq-manager-v1.0.0"
}
```